

Exercise 6 - ARPU Prediction

Engineer features from usage, tenure and plan data, train regression models, and interpret which drivers actually move revenue.

1. Objective

Predicting a continuous outcome changes the metrics, the diagnostics and the failure modes. In this exercise you will build a regression model for subscriber ARPU, read its residuals rather than only its R^2 , and translate the top drivers into something a marketing team could act on.

2. Prerequisites

- Exercise 5 completed - the pipeline pattern is reused here.
- The distinction between RMSE, MAE and R^2 from the Part 1 session.

3. Files required

File	Description
telecom_master.csv	The same subscriber master file used in MoP 4.1
Module_4_Data_Dictionary.docx	Note in particular how total_charges is derived

4. Procedure

Step 1 - Establish the target and remove its shadow

The target is arpu. One column in the file is very nearly arpu multiplied by tenure_months. Confirm this yourself before dropping it - the check is the point.

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt
df = pd.read_csv('telecom_master.csv')

implied = df['total_charges'] / df['tenure_months']
print('Correlation with arpu:', round(implied.corr(df['arpu']), 4))
print(df[['arpu']].join(implied.rename('implied_arpu')).head())
```

LEARNING POINT total_charges is not wrong data - it is perfectly valid for reporting. It is wrong *for this target*, because it contains the answer. Leakage is a property of the pairing of feature and target, not of the column on its own.

Step 2 - Engineer at least four features

Build features that encode behaviour, not just raw measurements. Include at least one ratio and one banded variable.

```
df['tenure_band'] = pd.cut(df['tenure_months'],
                           bins=[0, 6, 18, 36, 72],
                           labels=['0-6m', '7-18m', '19-36m', '37m+'])
df['data_per_voice'] = df['avg_monthly_gb'] / df['avg_voice_min'].clip(lower=1)
df['complaints_per_yr'] = df['complaints_6m'] * 2 / (df['tenure_months'] / 12).clip(lower=0.5)
df['is_heavy_data'] = (df['avg_monthly_gb'] > df['avg_monthly_gb'].quantile(0.75)).astype(int)
```

Add at least one feature of your own design and justify it in a comment - what behaviour does it capture, and why would it plausibly relate to revenue?

Step 3 - Train two regressors on identical splits

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

drop = ['arpu', 'customer_id', 'total_charges', 'churn']
X = df.drop(columns=drop)
y = df['arpu']
num = X.select_dtypes(include=np.number).columns.tolist()
cat = X.select_dtypes(exclude=np.number).columns.tolist()

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=42)
pre = ColumnTransformer([
    ('n', Pipeline([('i', SimpleImputer(strategy='median')), ('s', StandardScaler())]), num),
    ('c', Pipeline([('i', SimpleImputer(strategy='most_frequent')),
                    ('o', OneHotEncoder(handle_unknown='ignore'))]), cat)])

for name, reg in [('Linear', LinearRegression()),
                  ('Random forest', RandomForestRegressor(n_estimators=300, random_state=42))]:
    m = Pipeline([('pre', pre), ('reg', reg)]).fit(X_tr, y_tr)
    p = m.predict(X_te)
    print(f'{name:14s} RMSE {mean_squared_error(y_te, p)**0.5:7.2f}'
          f' MAE {mean_absolute_error(y_te, p):7.2f} R2 {r2_score(y_te, p):.3f}')
```

EXPECTED RMSE in the tens of rupees against a mean ARPU in the hundreds. If R^2 comes back above 0.98, a leaking column is still in your feature set - go back to Step 1.

Step 4 - Plot and interpret the residuals

```
model = Pipeline([('pre', pre), ('reg', RandomForestRegressor(
    n_estimators=300, random_state=42))]).fit(X_tr, y_tr)
pred = model.predict(X_te)
resid = y_te - pred

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].scatter(pred, resid, s=8, alpha=.4); ax[0].axhline(0, color='crimson')
ax[0].set_xlabel('Predicted ARPU'); ax[0].set_ylabel('Residual')
ax[1].scatter(X_te['tenure_months'], resid, s=8, alpha=.4); ax[1].axhline(0, color='crimson')
ax[1].set_xlabel('Tenure (months)')
plt.tight_layout(); plt.show()
```

Write one sentence under each plot. Look specifically for a funnel shape, for curvature, and for whether the errors are centred on zero across the whole range.

LEARNING POINT A funnel means the model is least reliable on high-ARPU subscribers - precisely the group whose revenue you most want to predict. R^2 will not tell you this. The residual plot will, in one glance.

Step 5 - Check the errors by segment

```
seg = X_te.assign(resid=resid.values)
print(seg.groupby('region')['resid'].agg(['mean', 'std', 'count']).round(2))
print(seg.groupby('plan_type')['resid'].agg(['mean', 'std', 'count']).round(2))
```

A mean residual that is clearly non-zero within a group means that group is being systematically under- or over-predicted. Say which groups, and by how much in rupees.

Step 6 - Rank the revenue drivers

```
from sklearn.inspection import permutation_importance
imp = permutation_importance(model, X_te, y_te, n_repeats=8, random_state=1)
names = X_te.columns
pd.Series(imp.importances_mean, index=names).sort_values(ascending=False).head(8).round(4)
```

Take the top two drivers and write a short recommendation: what should a marketing team do differently given that these move revenue? Keep it to five lines and avoid all model vocabulary.

CAUTION Importance is association, not cause. If heavy data usage ranks first, that does not prove that pushing data bundles raises ARPU - heavy users may simply be on richer plans already. Say so in your recommendation.

5. Expected outcomes

- **Leak check** - Evidence that total_charges leaks the target, with the correlation you computed.
- **Features** - At least four engineered features, one of your own design, each justified in a comment.
- **Comparison** - A metric comparison table on RMSE, MAE and R^2 for two regressors.
- **Diagnostics** - Two annotated residual plots and a segment-level error table.
- **Recommendation** - A five-line recommendation naming the top two drivers, in business language.

6. Good practices

- Report RMSE in rupees to a business audience. R^2 answers a question they did not ask.
- Always plot residuals before quoting a regression result. It takes one cell.
- Band a variable when the relationship is not plausibly linear, and say why you chose those cut points.
- Distinguish association from cause explicitly whenever you present importances.